

Министерство образования Республики Беларусь

Учреждение образования
**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ**

Факультет информационных технологий и управления

Кафедра вычислительных методов и программирования

Дисциплина: Основы алгоритмизации и программирования

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
курсового проекта
на тему

СИСТЕМА ОРГАНИЗАЦИИ ДВИЖЕНИЯ АВТОБУСОВ

БГУИР КП 6-05-0611-03 094 ПЗ

Студент

П.А. Кожевников

Руководитель

Е.В. Бегляк

Минск 2026

(На этом месте будет вставлен лист задания, который мы с вами заполняли и подписывали. Сейчас этот лист находится на подписи у заведующего каф.ВМиП.)

Шрифт в названиях и тексте пояснительной записки Times New Roman 14 пунктов, межстрочный интервал – 1,0 (или 18 пунктов), абзацный отступ – 1,25 см, выравнивание текста пояснительной записки – по ширине страницы.

В названиях разделов, подразделов, таблиц и рисунков не допускаются переносы в словах, в остальной части автоматические переносы в тексте пояснительной записки разрешены.

В тексте пояснительной записки обязательно должны быть указаны ссылки на литературные источники и ссылки на приложения.

Все остальные требования к пояснительной записке в СТП-2024.

На титульном листе и в ведомости документов обозначение пояснительной записки (БГУИР КП 6-05-0611-03 074 ПЗ) состоит из:

- БГУИР – код организации;
- КП – курсовой проект;
- 6-05-0611-03 – код специальности;
- 074 – номер темы задания;
- ПЗ – пояснительная записка.

СОДЕРЖАНИЕ

Введение	4
1 Структуры и файлы.....	5
1.1 Пользовательская структура.....	5
1.2 Структура стек	5
1.3 Файлы	5
2 Алгоритмы сортировки.....	8
2.1 Быстрая сортировка по пункту назначения	8
2.2 Сортировка вставками по времени прибытия	9
2.3 Сортировка выбором по времени отправления.....	9
3 Алгоритмы поиска.....	10
3.1 Линейный поиск.....	10
3.2 Бинарный поиск	11
4 Пользовательские функции	12
5 Описание работы программы	15
5.1 Работа с данными файла.....	15
5.3 Поиск рейсов по критериям.....	17
5.4 Сортировка данных по заданным критериям	18
5.5 Формирование отчетов	19
Заключение.....	21
Список использованных источников	22
Приложение А (обязательное) Листинг кода	23
Приложение Б (обязательное) Блок-схемы функций.....	34
Ведомость	47

ВВЕДЕНИЕ

На данный момент эффективная организация общественного транспорта, в частности автобусов, является ключевым аспектом транспортной инфраструктуры. Автобусные маршруты обеспечивают ежедневные передвижения миллионов людей, и от качества управления расписанием, маршрутами и парком транспортных средств напрямую зависит комфорт пассажиров. Автоматизированные системы обработки данных о движении автобусов позволяют значительно сократить время ожидания, оптимизировать загруженность маршрутов и своевременно реагировать на изменения в транспортной сети.

Целью курсового проекта является создание системы управления рейсами автовокзала, позволяющей оперативно обрабатывать данные о рейсах, фиксировать актуальное расписание и организовывать детальную информацию о маршрутах и автобусах. Программа обеспечивает добавление новых рейсов сразу после их появления на автовокзале, при этом фиксирует важнейшие параметры: номер рейса, тип автобуса, пункт назначения, время отправления и прибытия.

Актуальность выбранной темы обусловлена тем, что современные автовокзалы обрабатывают большое количество рейсов, и ручное ведение расписания становится неудобным и ненадёжным. Автоматизированная система позволяет упростить этот процесс и минимизировать количество ошибок.

Встроенные алгоритмы сортировки и поиска позволяют пользователям быстро находить подходящие рейсы по заданным критериям, таким как пункт назначения и время отправления. Дополнительно система предоставляет возможность просмотра статистических данных по рейсам, что позволяет отслеживать загруженность маршрутов и своевременно вносить изменения в расписание.

1 СТРУКТУРЫ И ФАЙЛЫ

1.1 Пользовательская структура

Структура (инициализация C/C++: `struct имя_структуры`) - это пользовательский тип данных, который объединяет несколько полей разных типов в единую сущность, которая позволяет хранить и управлять связанными данными[1].

В данном курсовом проекте для корректной работы использовалась структура, которая позволяет четко структурировать необходимые для решения поставленных задач данные, таковой структурой в проекте являются структура `System`.

Структура `System` объединяет следующие данные для каждой квартиры:

- `nomer` - номер рейса;
- `typeav[10]` - тип автобуса;
- `runnaz[15]` - пункт назначения;
- `vremotr[6]` - время отправления;
- `vrempri[6]` - время прибытия.

Данная структура содержит поля, позволяющие хранить всю необходимую информацию о рейсах.

Также была реализована структура для стека, используемого при итеративной реализации быстрой сортировки данных в бинарном файле.

1.2 Структура стек

Стек (`Stack`)— структура типа LIFO (Last In, First Out) – последним вошел, первым выйдет[2]. Элементы в стек можно добавлять или извлекать только через его вершину. Программно стек реализуется в виде однонаправленного списка с одной точкой входа – вершиной стека.

Стек хранит левую и правую границы участка файла (индексы элементов), который необходимо отсортировать, а также указатель на следующий элемент стека.

Структура `Stack`:

- `int low` - левая граница;
- `int high` - правая граница ;
- `Stack *next` - указатель на следующую ячейку;

Данная структура позволяет быстро извлекать левую и правую границы очередного участка файла для сортировки.

1.3 Файлы

Файл - база данных, представленная областью данных на каком-либо носителе информации. Благодаря использованию файлов программа может сохранять и загружать информацию даже после завершения работы. В данном проекте работа происходила непосредственно с бинарными файлами. При

разработке программного средства использовались стандартные функции для работы с файлами языка C в среде разработки VS Code:

- fopen() - открытие файла;
- fwrite() - запись данных в файл;
- fread() - чтение данных из файла;
- fseek() - перемещение файлового указателя на выбранную позицию;
- ftell() - возвращает позицию указателя в байтах;
- fclose() - закрытие файла.

В проекте применены методы работы с бинарными файлами, такие как: бинарная запись и чтение, добавление новых записей, загрузка данных при запуске программы, перезапись файла, а также обработка ошибок. Программа использует низкоуровневый подход к работе с файлами, применяя функции стандартной библиотеки C. Это позволяет эффективно сохранять и обрабатывать данные о рейсах в бинарном формате, ускоряя доступ к информации и минимизируя объем занимаемой памяти. При выполнении курсовой работы были созданы файлы, содержащие информацию о рейсах автовокзала: номер рейса, тип автобуса, пункт назначения, время отправления и прибытия.

1) Файл «data.bin» – бинарный файл, хранящий информацию о рейсах автовокзала (см. рисунок 1.1).

```
Number:67| Type:Minibus| Destination:Minsk| Departure time:12:45| Arrival time:13:34
Number:10| Type:Bus| Destination:Polotsk| Departure time:11:00| Arrival time:14:00
Number:12| Type:Avto| Destination:Turov| Departure time:23:00| Arrival time:01:00
Number:15| Type:Lodka| Destination:Myadel| Departure time:17:00| Arrival time:19:29
Number:17| Type:Electobus| Destination:Vitebsk| Departure time:07:00| Arrival time:09:09
Number:55| Type:Avto| Destination:Polotsk| Departure time:15:00| Arrival time:16:00
Number:99| Type:Bus| Destination:Minsk| Departure time:02:00| Arrival time:03:00
Number:100| Type:Minik| Destination:Turov| Departure time:22:00| Arrival time:00:00
Number:123| Type:Bus| Destination:Grodno| Departure time:12:45| Arrival time:16:00
```

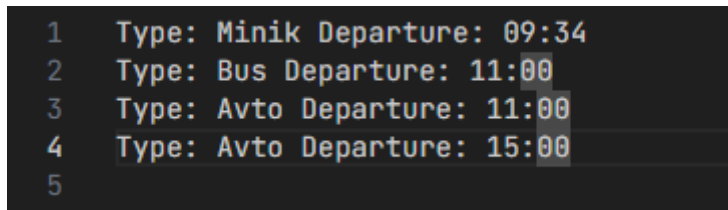
Рисунок 1.1 - Файл data.bin

2) Файл «buf.bin» – вспомогательный бинарный файл, используемый как буфер при сортировке (см. рисунок 1.2).

```
Number:67| Type:Minibus| Destination:Minsk| Departure time:12:45| Arrival time:13:34
Number:10| Type:Bus| Destination:Polotsk| Departure time:11:00| Arrival time:14:00
Number:12| Type:Avto| Destination:Turov| Departure time:23:00| Arrival time:01:00
Number:15| Type:Lodka| Destination:Myadel| Departure time:17:00| Arrival time:19:29
Number:17| Type:Electobus| Destination:Vitebsk| Departure time:07:00| Arrival time:09:09
Number:55| Type:Avto| Destination:Polotsk| Departure time:15:00| Arrival time:16:00
Number:99| Type:Bus| Destination:Minsk| Departure time:02:00| Arrival time:03:00
Number:100| Type:Minik| Destination:Turov| Departure time:22:00| Arrival time:00:00
Number:123| Type:Bus| Destination:Grodno| Departure time:12:45| Arrival time:16:00
Now you are at buf.bin
```

Рисунок 1.2 - Файл buf.bin

3)Файл «result.txt» – вспомогательный бинарный файл, используемый как буфер при сортировке (см. рисунок 1.3).



```
1 Type: Minik Departure: 09:34
2 Type: Bus Departure: 11:00
3 Type: Avto Departure: 11:00
4 Type: Avto Departure: 15:00
5
```

Рисунок 1.3 - Файл result.txt

Таким образом, использование структур и файлов в данном проекте обеспечивает надёжную основу для хранения и обработки информации о рейсах автовокзала: структура *System* позволяет логически объединить все атрибуты рейса в единую сущность, структура *Stack* обеспечивает эффективную итеративную сортировку данных, а бинарные файлы «data.bin» и «buf.bin» гарантируют сохранность информации между сеансами работы программы и ускоряют доступ к ней, тогда как файл «result.txt» предоставляет удобный способ вывода итоговых данных пользователю.

2 АЛГОРИТМЫ СОРТИРОВКИ

Сортировка - это процесс упорядочивания элементов в некотором массиве или коллекции данных по определенному правилу или критерию[3]. В общем случае сортировка используется для того, чтобы упростить обработку данных, сделать поиск более эффективным и улучшить визуальное восприятие.

Ключ сортировки - это критерий, по которому происходит упорядочивание элементов. Это значение или атрибут, который используется для сравнения элементов в процессе сортировки.

В данном курсовом проекте были успешно реализованы алгоритмы быстрой сортировки (по пункту назначения), сортировки выбором (по времени отправления) и сортировки вставками (по времени прибытия), адаптированные для прямой работы с последовательными структурами данных внутри бинарных файлов посредством произвольного доступа.

2.1 Быстрая сортировка по пункту назначения

Быстрая сортировка Хоара (QuickSort) — один из самых эффективных алгоритмов сортировки, основанный на стратегии «разделяй и властвуй»[4]. Алгоритм быстрой сортировки был придуман Тони Хоаром, в среднем он выполняется за $n \cdot \log(n)$ шагов. В худшем случае сложность опускается до порядка n^2 , хотя такая ситуация довольно редкая. Быстрая сортировка обычно быстрее других "скоростных" сортировок со сложностью порядка $n \cdot \log(n)$. Быстрая сортировка не устойчивая. Обычно, она преподносится как рекурсивная, однако, в данном курсовом проекте была реализована с помощью стека при этом потребуется не более $n \cdot \log(n)$ дополнительной памяти [7].

Существует два способа реализации итеративной версии сортировки: хранение левой и правой границ в одной ячейке стека либо использование двух параллельных стеков границ[7]. Поскольку второй вариант требует большего объёма памяти и является менее эффективным, в данной реализации выбран первый подход - совместное хранение границ.

Алгоритм работы сортировки:

- выбрать опорный элемент (pivot) - крайний левый элемент текущего подотрезка, считанный из файла с помощью fseek и fread;
- с помощью двух указателей l и r, движущихся навстречу друг другу, сравнить все остальные элементы с опорным, считывая каждый из них из файла, и переставить так, чтобы слева от опорного оказались элементы меньшие, справа - большие;
- после того как указатели пересеклись, опорный элемент помещается на своё окончательное место с помощью swap, которая переставляет записи непосредственно в файле;

- для левой и правой частей выполнить ту же последовательность операций итеративно, используя явный стек, хранящий границы подотрезков, если длина отрезка больше единицы.

В данном случае функция принимает указатель на файл, чтобы после каждой перестановки записывать результат непосредственно на диск с помощью `fseek` и `fwrite`. Вместо рекурсии используется явный стек, хранящий пары границ (`left`, `right`) необработанных подотрезков, что исключает переполнение стека вызовов на больших файлах. Фрагмент кода программы (см. Приложение А стр.26).

2.2 Сортировка вставками по времени прибытия

Сортировка вставками (`Insertion Sort`) - это один из простейших алгоритмов сортировки, который работает аналогично тому, как люди сортируют карты в руках. Алгоритм проходит по файлу и вставляет каждый элемент на его правильное место среди уже отсортированных элементов.

Принцип работы алгоритма:

- начинаем со второй записи (считаем, что первая уже отсортирована), считывая её из файла с помощью `fseek` и `fread`;
- берём следующую запись и сравниваем её с уже отсортированными, сдвигая записи вправо с помощью `fseek` и `fwrite` до тех пор, пока не найдено правильное место;
- записываем текущий элемент на найденную позицию непосредственно в файл;
- повторяем, пока все записи файла не будут отсортированы.

Временная сложность алгоритма в среднем случае равна $O(n^2)$. Особенностью данной сортировки является простая реализация и хорошая производительность на уже частично отсортированных данных - если в файле окажется один смещённый элемент, алгоритм быстро поставит его на место. Все операции чтения и записи производятся непосредственно в бинарном файле через `fseek`, `fread` и `fwrite`. Фрагмент кода программы (см. Приложение А стр.26).

2.3 Сортировка выбором по времени отправления

Сортировка выбором (`Selection Sort`) - это простой алгоритм сортировки, который работает по принципу нахождения минимального элемента в неотсортированной части файла и перемещения его в начало отсортированной части. По времени выполнения равен сортировке «пузырьком». При сортировке предполагается, что сравнения делаются за постоянное время.

Принцип работы алгоритма:

- начинаем с первой записи, считывая её из файла через `fseek` и `fread`;
- находим минимальную запись в неотсортированной части файла, последовательно считывая и сравнивая записи;

- меняем её местами с первой записью неотсортированной части через swap;

- повторяем процесс для оставшейся части файла, каждый раз увеличивая размер отсортированной части на одну запись.

Все операции чтения и записи производятся непосредственно в бинарном файле через fseek, fread и fwrite. Фрагмент кода программы (см. Приложение А стр.27).

3 АЛГОРИТМЫ ПОИСКА

Алгоритмы поиска - это методы нахождения нужного элемента в массиве, списке, дереве или другой структуре данных[5]. Они помогают быстро определить, присутствует ли искомый элемент в коллекции, и если да, то, где именно.

Существуют такие алгоритмы поиска как:

- линейный поиск;
- бинарный поиск;
- интерполяционный поиск;
- поиск в глубину (DFS) и ширину (BFS).

В курсовом проекте представлены два алгоритма поиска: линейный и бинарный поиски.

3.1 Линейный поиск

Линейный поиск (Linear Search) - это самый простой, но медленный метод поиска, который просто перебирает все элементы массива, либо любой другой базы данных (хранилища информации), по порядку, пока не найдет нужный.

Алгоритм работы поиска:

- открываем бинарный файл и считываем записи последовательно через fseek и fread;
- сравниваем поле искомой записи с заданным значением через strcmp;
- если совпадает - выводим найденную запись на экран и продолжаем поиск, так как записей с одинаковым значением может быть несколько;
- если нет - движемся к следующей записи;
- если дошли до конца файла и не нашли ни одного совпадения - выводим сообщение об отсутствии элемента.

Временная сложность данного поиска оставляет желать лучшего и равна n - единственный минус данного метода поиска.

Из плюсов можно выделить лишь простую реализацию и то, что он работает на неотсортированных данных, в отличие от бинарного поиска. Фрагмент кода программы (см. Приложение А стр.28).

3.2 Бинарный поиск

Бинарный поиск - это эффективный алгоритм поиска, который работает, в отличие от линейного поиска, только на отсортированных заранее данных. Вместо перебора всех элементов, он разделяет массив пополам и исключает ненужную часть, значительно ускоряя поиск.

Принцип работы:

- копирует записи из основного файла в буферный;
- перед поиском буферный файл сортируется по искомому полю, так как бинарный поиск требует отсортированных данных;
- вычисляем середину файла - считываем запись через `fseek` и `fread` по позиции `mid * sizeof(System)`;
- сравниваем поле средней записи с искомым значением через `strcmp`: - если равно - запись найдена, выводим её на экран;
- если меньше - ищем в правой половине файла;
- если больше - ищем в левой половине файла;
- повторяем процесс, сужая границы `left` и `right`, пока не найдём запись или границы не пересекутся.

Для реализации данного поиска в курсовой работе был разработан алгоритм бинарного поиска, работающий непосредственно с бинарным файлом через `fseek` и `fread`, идущий совместно с функцией вывода найденных значений на экран пользователю[6]. Поскольку бинарный поиск находит только одно совпадение, после нахождения элемента производится дополнительный проход по соседним записям для вывода всех возможных совпадений. Фрагмент кода программы (см. Приложение А стр.29).

4 ПОЛЬЗОВАТЕЛЬСКИЕ ФУНКЦИИ

Функция `swap()` - принимает указатель на файл и два индекса `i` и `j`. Функция отвечает за перестановку двух записей в бинарном файле - считывает записи с позиций `i` и `j` через `fseek` и `fread`, после чего записывает их обратно в обратном порядке через `fwrite`. Функция используется в алгоритмах сортировки для обмена элементов местами, что позволяет изменять порядок записей непосредственно внутри бинарного файла без загрузки всего файла в оперативную память.

Функция `copy_file()` - принимает два пути к файлам: источник и приёмник. Функция побайтово копирует содержимое бинарного файла-источника в файл-приёмник, считывая записи через `fread` и записывая через `fwrite`. Используется перед каждой сортировкой для создания рабочей копии файла. Это позволяет сохранять исходный файл без изменений и выполнять сортировку или поиск на временной копии данных.

Функция `dobav()` - принимает путь к файлу. Функция отвечает за добавление новой записи в конец бинарного файла - открывает файл в режиме дозаписи "ab", считывает с клавиатуры данные о маршруте и записывает их в файл через `fwrite`. Использование режима дозаписи позволяет не перезаписывать существующие данные, а расширять файл новыми записями.

Функция `ydalen()` - принимает путь к файлу. Функция отвечает за удаление записи из бинарного файла по номеру маршрута - находит нужную запись последовательным перебором, сдвигает все последующие записи на одну позицию влево через `fseek`, `fread` и `fwrite`, после чего усекает файл на одну запись с помощью `chsize`. Такой подход позволяет физически удалить запись из файла и сохранить целостность структуры данных.

Функция `izme()` - принимает путь к файлу. Функция отвечает за изменение полей существующей записи - находит запись по номеру маршрута, предлагает пользователю выбрать поле для изменения и записывает обновлённую запись обратно в файл через `fseek` и `fwrite`. Это позволяет редактировать отдельные записи без необходимости пересоздания всего файла.

- Функция `poiskpriz()` - принимает пути к основному файлу, буферному файлу и текстовому файлу результатов. Функция выполняет поиск по признаку - предварительно сортирует файл функцией `viborsort` по времени отправления, затем последовательно перебирает все записи, отбирая те, у которых пункт назначения совпадает с заданным и время прибытия не превышает указанного. Пользователь может выбрать способ отображения результатов: вывод в консоль или сохранение в текстовый файл. При выборе вывода в консоль найденные маршруты отображаются на экране в порядке возрастания времени отправления, при выборе файла - результаты сохраняются в текстовый файл через `fprintf`. Использование сортировки обеспечивает упорядоченное представление данных, а гибкий выбор формата

вывода позволяет сохранять результаты поиска в удобном для просмотра и дальнейшего анализа виде.

- Функция `staticiok()` - принимает пути к основному файлу и текстовому файлу результатов. Функция выполняет статистический отбор - перебирает все записи файла и отбирает те, у которых тип транспорта совпадает с заданным и время отправления не раньше указанного. Пользователь может выбрать способ отображения результатов: вывод в консоль или сохранение в текстовый файл. При выборе вывода в консоль результаты отображаются на экране построчно, при выборе файла - найденные записи сохраняются в текстовый файл через `fprintf`. В обоих случаях выводится общее количество найденных записей, соответствующих критериям поиска. Это позволяет выполнять гибкий анализ данных и получать статистическую информацию по заданным критериям с удобным выбором формата представления результатов.

- Функция `pros()` - принимает путь к файлу. Функция отвечает за вывод всех записей бинарного файла на экран - последовательно считывает каждую запись через `fread` и выводит все поля в форматированном виде через `cout`. Последовательное чтение файла позволяет просматривать всё содержимое базы данных независимо от количества записей.

Функция `create()` - принимает путь к файлу. Функция отвечает за создание нового бинарного файла и запись в него данных о маршрутах - открывает файл в режиме записи "`wb`", последовательно считывает введенные пользователем данные и сохраняет их в файл через `fwrite`. Функция используется для первоначального формирования базы данных маршрутов, что позволяет создавать новый файл и заносить в него необходимую информацию о транспортных записях.

Функция `push()` — принимает указатель на вершину стека `top` и два значения `low` и `high`. Функция отвечает за добавление нового элемента в стек: выделяет память под новый узел, записывает в него значения `low` и `high`, после чего связывает новый элемент с предыдущей вершиной стека через указатель `next`. В результате функция возвращает указатель на новую вершину стека. Функция используется для сохранения границ диапазонов, например в алгоритмах сортировки или обработки данных, где требуется временно хранить пары значений.

Функция `pop()` — принимает указатель на вершину стека `top`, а также указатели `low` и `high` для возврата данных удаляемого элемента. Функция отвечает за извлечение элемента из стека: проверяет, не пуст ли стек, считывает значения `low` и `high` из вершины, сохраняет указатель на следующий элемент, освобождает память текущего узла и возвращает указатель на новую вершину стека. Функция используется для получения ранее сохранённых диапазонов или данных в порядке LIFO (последний вошёл — первый вышел).

Функция `proverka()` — принимает имя файла `filename` и режим открытия `mode`. Функция отвечает за безопасное открытие файла: выполняет попытку открыть файл с помощью `fopen`, после чего проверяет успешность

операции. Если файл открыть не удалось, функция выводит сообщение об ошибке "Dont open" и возвращает NULL. В случае успешного открытия возвращается указатель на файл типа FILE*. Функция используется для упрощения работы с файлами и предотвращения ошибок, связанных с попыткой обращения к неоткрытому файлу.

Таким образом, разработанный набор пользовательских функций обеспечивает полный цикл работы с бинарными файлами и записями маршрутов: создание базы данных, добавление, удаление, изменение, поиск, сортировку и вывод информации. Использование файловых операций `fopen`, `fread`, `fwrite`, `fseek` и других позволило организовать эффективную обработку данных непосредственно в бинарном файле без необходимости полной загрузки содержимого в оперативную память. Дополнительные функции работы со стеком обеспечивают поддержку алгоритмов сортировки и обработки диапазонов данных. Разделение программы на отдельные функции повысило её модульность, упростило сопровождение кода и сделало структуру программы более понятной и удобной для дальнейшего расширения.

5 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ

5.1 Работа с данными файла

Описание работы программы подразумевает описание результатов работы подпрограмм, позволяющих пользователю выполнять обработку и управление данными, хранящимися в бинарном файле.

Функции `dobav()`, `ydalen()`, `pros()`, `izme()` позволяют пользователю выполнять основные операции с записями в бинарном файле: добавление, удаление, просмотр и изменение данных. Они обеспечивают удобное управление структурированной информацией без необходимости ручного редактирования файла.

На рисунке 5.1 представлена функция `dobav()`. В этой функции пользователь вводит данные о новом рейсе: номер рейса, тип автобуса, пункт назначения, время отправления и прибытия на конечный пункт. Далее за счет режима работы файла “ab” все старые записи сохраняются, а новая запись добавляется в конец.

```
Example: 25 Bmw London 12:45 13:09
Enter data :
  number :
23
Type
Bus
Destination
Polotsk
Departure time
12:00
Arrival time
14:00
more?
1-NO
```

Рисунок 5.1 – Добавить рейс

На рисунке 5.2 представлена функция `ydaLen()`. В этой функции пользователь вводит номер рейса, который необходимо удалить. После ввода номера программа считывает данные до тех пор, пока не найдет нужную запись. После нахождения записи выполняется сдвиг элемента в конец файла за счет замены мест с соседним элементом, а затем с помощью функции `chsize()` уменьшается размер исходного файла.

```
7. Statistics
10. Delete
11. Added
12. Change
13. New file
0. Exit
Your choice: 10
Vvedite nomer marshruta: 67
```

Рисунок 5.2 – Удалить рейс

На рисунке 5.3 представлена функция `izme()`. В этой функции пользователь вводит номер рейса, который хочет изменить. После ввода номера программа проходит по файлу до тех пор, пока не найдет нужную запись. После нахождения записи программа спрашивает у пользователя, какое поле он хочет изменить.

```
Enter number:
17
What to change:
1-Number :
2-Type
3-Destination
4-Departure time
5-Arrival time
2
Enter new type:
Polo
```

Рисунок 5.3 – Изменение данных рейса

На рисунке 5.4 представлена функция `pros()`. Функция выводит запись из файла при успешном ее считывании.

```
Your choice: 2
Number:100| Type:Minik| Destination:Turov| Departure time:22:00| Arrival time:00:00
Number:12| Type:Avto| Destination:Turov| Departure time:23:00| Arrival time:01:00
Number:17| Type:Polo| Destination:Vitebsk| Departure time:07:00| Arrival time:09:09
Number:6767| Type:Car| Destination:Rubleushina| Departure time:06:00| Arrival time:10:00
Number:999| Type:Minik| Destination:Polotsk| Departure time:09:34| Arrival time:10:27
Number:10| Type:Bus| Destination:Polotsk| Departure time:11:00| Arrival time:14:00
Number:567| Type:Avto| Destination:Myadel| Departure time:11:00| Arrival time:15:00
Number:123| Type:Bus| Destination:Grodno| Departure time:12:45| Arrival time:16:00
Number:55| Type:Avto| Destination:Polotsk| Departure time:15:00| Arrival time:16:00
Number:589| Type:Minibus| Destination:Turov| Departure time:17:08| Arrival time:19:06
Number:15| Type:Lodka| Destination:Myadel| Departure time:17:00| Arrival time:19:29
```

Рисунок 5.4 – Просмотр записей

5.3 Поиск рейсов по критериям

Поиск позволяет пользователю быстро найти нужный рейс по заданным критериям. Это упрощает работу с файлом и ускоряет получение необходимой информации. Функции `vremotp()`, `punnaz()` позволяют пользователю выполнить поиск рейсов по времени отправления или пункту назначения.

На рисунке 5.5 продемонстрирована функция `vremotp()`. В основе данной функции лежит линейный поиск. Программа запрашивает у нас время отправления, затем при нахождении нужного времени выводит данные о рейсе. Поиск идет до тех пор, пока данные с файла успешно считываются.

```
Enter departure time
11:00
Found:
Destination:Polotsk| Number:10| Type:Bus| Arrival time:14:00
Found:
Destination:Myadel| Number:567| Type:Avto| Arrival time:15:00
```

Рисунок 5.5 – Поиск рейсов

На рисунке 5.6 показан результат работы функции `punnaz()`. В основе данной функции лежит бинарный. В начале программа переносит данные из основного файла в буферный файл для их сортировки, без которой реализация поиска невозможна. Затем программа берёт элемент из середины и, если он соответствует условию, от найденного элемента запускает два линейных поиска для нахождения остальных записей.

```
Your choice: 4
Enter destination
Myadel
Result:
Number:567| Type:Avto| Departure time:11:00| Arrival time:15:00
Number:15| Type:Lodka| Departure time:17:00| Arrival time:19:29
```

Рисунок 5.6 – Поиск рейсов

5.4 Сортировка данных по заданным критериям

Сортировка позволяет упорядочить данные по заданным критериям. Она облегчает просмотр записей и последующую работу с ними. Функции `quicksorts()`, `vstafsorts()`, `viborsort()` выполняют сортировку по определенным критериям.

На рисунке 5.7 показана функция `quicksorts()`. Данная функция сортирует записи по пункту назначения, с использованием стека для хранения границ подмассивов. Алгоритм выбирает опорный элемент, разделяет файл на две части (меньшие слева, большие справа) и повторяет процесс для каждой части до полной сортировки.

```
Your choice: 6
Number:123| Type:Bus| Destination:Grodno| Departure time:12:45| Arrival time:16:00
Number:15| Type:Lodka| Destination:Myadel| Departure time:17:00| Arrival time:19:29
Number:567| Type:Avto| Destination:Myadel| Departure time:11:00| Arrival time:15:00
Number:55| Type:Avto| Destination:Polotsk| Departure time:15:00| Arrival time:16:00
Number:999| Type:Minik| Destination:Polotsk| Departure time:09:34| Arrival time:10:27
Number:10| Type:Bus| Destination:Polotsk| Departure time:11:00| Arrival time:14:00
Number:6767| Type:Car| Destination:Rubleushina| Departure time:06:00| Arrival time:10:00
Number:100| Type:Minik| Destination:Turov| Departure time:22:00| Arrival time:00:00
Number:12| Type:Avto| Destination:Turov| Departure time:23:00| Arrival time:01:00
Number:589| Type:Minibus| Destination:Turov| Departure time:17:08| Arrival time:19:06
Number:17| Type:Polo| Destination:Vitebsk| Departure time:07:00| Arrival time:09:09
```

Рисунок 5.7 – Сортировка записей

На рисунке 5.8 показана функция `viborsort()`. Данная функция сортирует записи по времени отправления методом выбора, находя минимальный элемент в неотсортированной части файла и меняя его местами с первым элементом этой части. Процесс повторяется для оставшихся элементов до полной сортировки файла.

```
Your choice: 2
Number:6767| Type:Car| Destination:Rubleushina| Departure time:06:00| Arrival time:10:00
Number:17| Type:Polo| Destination:Vitebsk| Departure time:07:00| Arrival time:09:09
Number:999| Type:Minik| Destination:Polotsk| Departure time:09:34| Arrival time:10:27
Number:567| Type:Avto| Destination:Myadel| Departure time:11:00| Arrival time:15:00
Number:10| Type:Bus| Destination:Polotsk| Departure time:11:00| Arrival time:14:00
Number:123| Type:Bus| Destination:Grodno| Departure time:12:45| Arrival time:16:00
Number:55| Type:Avto| Destination:Polotsk| Departure time:15:00| Arrival time:16:00
Number:15| Type:Lodka| Destination:Myadel| Departure time:17:00| Arrival time:19:29
Number:589| Type:Minibus| Destination:Turov| Departure time:17:08| Arrival time:19:06
Number:100| Type:Minik| Destination:Turov| Departure time:22:00| Arrival time:00:00
Number:12| Type:Avto| Destination:Turov| Departure time:23:00| Arrival time:01:00
```

Рисунок 5.8 – Сортировка записей

На рисунке 5.9 показана функция `vstafsort()`. Данная функция сортирует записи по времени прибытия методом вставки, последовательно вставляя каждый элемент в правильную позицию среди уже отсортированных элементов путём сдвига больших элементов вправо. Алгоритм обрабатывает элементы слева направо, поддерживая отсортированную часть файла в начале.

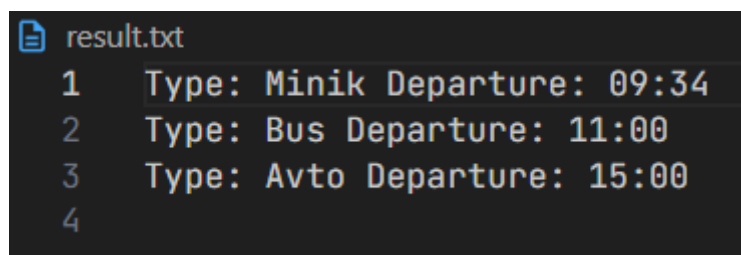
```
Your choice: 2
Number:100| Type:Minik| Destination:Turov| Departure time:22:00| Arrival time:00:00
Number:12| Type:Avto| Destination:Turov| Departure time:23:00| Arrival time:01:00
Number:17| Type:Polo| Destination:Vitebsk| Departure time:07:00| Arrival time:09:09
Number:6767| Type:Car| Destination:Rubleushina| Departure time:06:00| Arrival time:10:00
Number:999| Type:Minik| Destination:Polotsk| Departure time:09:34| Arrival time:10:27
Number:10| Type:Bus| Destination:Polotsk| Departure time:11:00| Arrival time:14:00
Number:567| Type:Avto| Destination:Myadel| Departure time:11:00| Arrival time:15:00
Number:123| Type:Bus| Destination:Grodno| Departure time:12:45| Arrival time:16:00
Number:55| Type:Avto| Destination:Polotsk| Departure time:15:00| Arrival time:16:00
Number:589| Type:Minibus| Destination:Turov| Departure time:17:08| Arrival time:19:06
Number:15| Type:Lodka| Destination:Myadel| Departure time:17:00| Arrival time:19:29
```

Рисунок 5.9 – Сортировка записей

5.5 Формирование отчетов

Функционал программы предусмотрел создание текстовых отчетов на основе заданных критериев фильтрации данных из бинарного файла. Функция `roiskpriz()` формирует отчет с рейсами, прибывающими в указанный пункт назначения не позднее заданного времени, а функция `staticiok()` создает отчет с подсчетом количества рейсов определенного типа транспорта, отправляющихся после указанного времени.

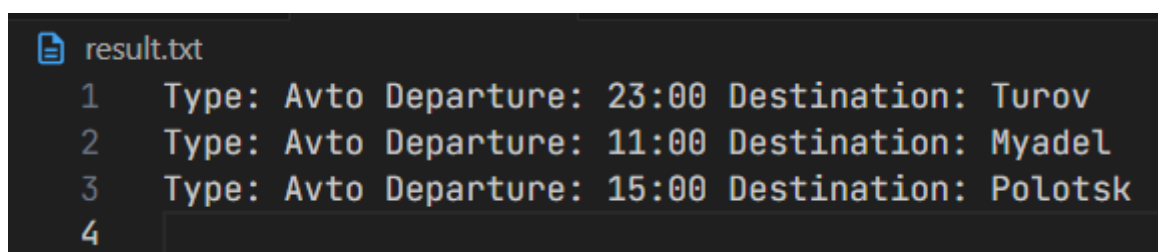
На рисунке 5.10 показана функция `poiskpriz()`. Данная функция предоставляет пользователю выбор между выводом результатов в консоль или формированием текстового отчета с рейсами, которые прибывают в указанный пункт назначения не позднее заданного времени. Функция предварительно сортирует данные методом выбора по времени отправления для обеспечения упорядоченного представления результатов. При выборе вывода в файл результаты записываются в текстовый документ с указанием типа транспорта и времени отправления. При выборе консольного вывода информация отображается на экране в том же формате, что обеспечивает гибкость в работе с результатами поиска.



```
result.txt
1  Type: Minik Departure: 09:34
2  Type: Bus Departure: 11:00
3  Type: Avto Departure: 15:00
4
```

Рисунок 5.10 – Поиск по признаку

На рисунке 5.11 показана функция `staticiok()`. Данная функция предоставляет пользователю выбор между выводом результатов в консоль или формированием текстового отчета с маршрутами, которые соответствуют заданному типу транспорта и отправляются не ранее указанного времени. Функция выполняет последовательный перебор всех записей файла, отбирая данные по заданным критериям. При выборе вывода в файл результаты записываются в текстовый документ с указанием типа транспорта, времени отправления и пункта назначения, а также общего количества найденных маршрутов. При выборе консольного вывода информация отображается на экране в том же формате, что обеспечивает гибкость в работе со статистическими данными и их анализом.



```
result.txt
1  Type: Avto Departure: 23:00 Destination: Turov
2  Type: Avto Departure: 11:00 Destination: Myadel
3  Type: Avto Departure: 15:00 Destination: Polotsk
4
```

Рисунок 5.11 – Статистика

Функционал программы включает базовые операции работы с данными: функция `pros()` обеспечивает просмотр всех записей файла, `dobav()`

добавляет новую запись в конец файла, `izme()` позволяет изменить любое поле выбранной записи по номеру рейса, а `ydalen()` удаляет запись со сдвигом всех последующих элементов. Для поиска реализованы функции `vremotp()`, выполняющая линейный поиск по времени отправления, и `rupnaz()`, использующая бинарный поиск по пункту назначения после предварительной сортировки файла методом быстрой сортировки. Формирование отчетов осуществляется функциями `poiskpriz()` и `staticiok()`, которые фильтруют данные по заданным критериям и сохраняют результаты в текстовые файлы с подсчетом статистики.

ЗАКЛЮЧЕНИЕ

По результатам работы над курсовым проектом сформирована следующая характеристика: разработано работающее программное средство, позволяющее создавать базу данных маршрутов общественного транспорта, просматривать и удалять записи, а также вносить изменения и получать информацию о маршрутах с использованием различных методов сортировки и поиска:

разработчик продемонстрировал уверенное владение методами работы с бинарными файлами, структурами и другими необходимыми средствами, что является залогом высокой эффективности созданной системы;

было создано программное средство, обладающее широким функционалом и возможностями. Система позволяет удобно управлять информацией о маршрутах, что существенно упрощает процесс поиска и анализа расписания;

реализован интуитивно понятный и функциональный консольный интерфейс, обеспечивающий быстрый и лёгкий доступ к основным операциям приложения. Пользователь может оперативно находить нужные маршруты, редактировать данные, а также формировать отчёты по заданным критериям;

при разработке использовались алгоритмы быстрой сортировки, сортировки выбором, сортировки вставками, а также линейный и бинарный поиск, что позволяет системе обрабатывать большие объёмы данных, обеспечивать стабильную и надёжную работу;

итогом разработки можно назвать успешное создание многофункционального программного средства, готового к корректной работе в сфере управления расписанием маршрутов общественного транспорта.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Герберт Шилдт : С++ базовый курс [Электронный ресурс]. – Режим доступа: <https://wow-only.ru/books/shield.pdf>. – Дата доступа: 14.02.2026.
- [2] Герберт Шилдт : С++ базовый курс [Электронный ресурс]. – Режим доступа: <https://wow-only.ru/books/shield.pdf>. – Дата доступа: 14.02.2026.
- [3] Основы алгоритмизации и программирования. Язык Си : учеб. пособие / М. П. Батура, В.Л. Бусько, А.Г. Корбит, Т.М. Кривоносова - Минск : БГУИР, 2007. - 240 с
- [4] Основы алгоритмизации и программирования (язык С/С++). лабораторный практикум в 2 ч. Ч. 2 : учеб. – метод. пособие / Беспалов С. А. [и др.] . – Минск: БГУИР, 2018. – 114 с.
- [5] Побегайло, А. С/С++ для студента / А. Побегайло. – Санкт-Петербург : БХВ-Петербург, 2010. – 528 с.
- [6] Культин, Н. С/С++ в задачах и примерах / Н. Культин. – Санкт-Петербург : БХВ-Петербург, 2015. – 288 с.
- [7] Learn info [Электронный ресурс]. – Режим доступа: <https://learn.info/algorithms/quicksort.html>. – Дата доступа: 18.05.2026.

ПРИЛОЖЕНИЕ А

(Обязательное)

Листинг кода

```
#include <iostream>
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <io.h>
using namespace std;
struct System
{
    int nomer;
    char typeav[10];
    char punnaz[15];
    char vremotp[6];
    char vrempri[6];
};
struct Stack
{
    int low;
    int high;
    Stack *next;
};
FILE *proverka(const char *filename, const char *mode)
{
    FILE *f = fopen(filename, mode);
    if (f == NULL)
    {
        cout << "Dont open\n";
        return NULL;
    }
    return f;
}

Stack *push(Stack *top, int low, int high)
{
    Stack *ptr = new Stack;
    ptr->low = low;
    ptr->high = high;
    ptr->next = top;
    return ptr;
}

Stack *pop(Stack *top, int *low, int *high)
{
    if (top == NULL)
        return NULL;
    *low = top->low;
    *high = top->high;
    Stack *t = top->next;
    delete top;
    return t;
}

void swap(FILE *p, int i, int j)
{
    System a, b;
    fseek(p, i * sizeof(System), SEEK_SET);
    fread(&a, sizeof(System), 1, p);
    fseek(p, j * sizeof(System), SEEK_SET);
    fread(&b, sizeof(System), 1, p);
    fseek(p, i * sizeof(System), SEEK_SET);
```

```

        fwrite(&b, sizeof(System), 1, p);
        fseek(p, j * sizeof(System), SEEK_SET);
        fwrite(&a, sizeof(System), 1, p);
    }
}

void copy_file(const char *src, const char *dst)
{
    FILE *s = fopen(src, "rb");
    FILE *d = fopen(dst, "wb");
    System buf;
    while (fread(&buf, sizeof(System), 1, s) == 1)
        fwrite(&buf, sizeof(System), 1, d);
    fclose(s);
    fclose(d);
}

int ydalen(const char *filename)
{
    FILE *p = proverka(filename, "rb+");
    if (!p)
        return 1;
    int route;
    cout << "Vvedite nomer marshruta: ";
    cin >> route;
    System s;
    fseek(p, 0, SEEK_END);
    int n = ftell(p) / sizeof(System);
    int found = -1;
    for (int i = 0; i < n; i++)
    {
        fseek(p, i * sizeof(System), SEEK_SET);
        fread(&s, sizeof(s), 1, p);
        if (s.nomer == route)
        {
            found = i;
            break;
        }
    }
    if (found == -1)
    {
        cout << "Not found";
        fclose(p);
        return 1;
    }
    for (int i = found; i < n - 1; i++)
    {
        fseek(p, (i + 1) * sizeof(System), SEEK_SET);
        fread(&s, sizeof(s), 1, p);
        fseek(p, i * sizeof(System), SEEK_SET);
        fwrite(&s, sizeof(s), 1, p);
    }
    chsize(fileno(p), (n - 1) * sizeof(System));
    fclose(p);
    return 0;
}

int izme(const char *filename)
{
    FILE *p = proverka(filename, "rb+");
    if (!p)
        return 1;
    int buf, chos;
    cout << "Enter number: " << endl;
    cin >> buf;
    System s;

```



```

int i = 0;
while (fread(&s, sizeof(s), 1, p) == 1)
{
    if (s.nomer == buf)
    {
        cout << "What to change: " << endl;
        cout << "1-Number :" << endl;
        cout << "2-Type" << endl;
        cout << "3-Destination" << endl;
        cout << "4-Departure time" << endl;
        cout << "5-Arrival time" << endl;
        cin >> chos;
        switch (chos)
        {
            case 1:
            {
                cout << "Enter new number: " << endl;
                cin >> s.nomer;
                fseek(p, -(long)sizeof(System), SEEK_CUR);
                fwrite(&s, sizeof(System), 1, p);
                break;
            }
            case 2:
            {
                cout << "Enter new type: " << endl;
                cin.ignore();
                cin.getline(s.typeav, 10);
                fseek(p, -(long)sizeof(System), SEEK_CUR);
                fwrite(&s, sizeof(System), 1, p);
                break;
            }
            case 3:
            {
                cout << "Enter new destination: " << endl;
                cin.ignore();
                cin.getline(s.punnaz, 15);
                fseek(p, -(long)sizeof(System), SEEK_CUR);
                fwrite(&s, sizeof(System), 1, p);
                break;
            }
            case 4:
            {
                cout << "Enter new departure time: " << endl;
                cin.ignore();
                cin.getline(s.vremotp, 6);
                fseek(p, -(long)sizeof(System), SEEK_CUR);
                fwrite(&s, sizeof(System), 1, p);
                break;
            }
            case 5:
            {
                cout << "Enter new arrival time: " << endl;
                cin.ignore();
                cin.getline(s.vrempri, 6);
                fseek(p, -(long)sizeof(System), SEEK_CUR);
                fwrite(&s, sizeof(System), 1, p);
                break;
            }
            default:
            {
                cout << "not correct choose";
                return 0;
            }
        }
    }
}

```

```

    }
    fclose(p);
    return 0;
}
void quicksorts(const char *filename)
{
    FILE *g = proverka(filename, "rb+");
    if (!g)
        return;
    fseek(g, 0, SEEK_END);
    Stack *top = NULL;
    int left, right;
    int l, r;
    System bufr, bufl, pivot;
    top = push(top, 0, (ftell(g) / sizeof(System)) - 1);
    while (top != NULL)
    {
        top = pop(top, &left, &right);
        l = left + 1;
        r = right;
        fseek(g, left * sizeof(System), SEEK_SET);
        fread(&pivot, sizeof(System), 1, g);
        while (l <= r)
        {
            fseek(g, l * sizeof(System), SEEK_SET);
            fread(&bufl, sizeof(System), 1, g);
            fseek(g, r * sizeof(System), SEEK_SET);
            fread(&bufr, sizeof(System), 1, g);
            while (l <= r && strcmp(bufl.punnaz, pivot.punnaz) < 0)
            {
                l++;
                fseek(g, l * sizeof(System), SEEK_SET);
                fread(&bufl, sizeof(System), 1, g);
            }
            while (l <= r && strcmp(bufr.punnaz, pivot.punnaz) >= 0)
            {
                r--;
                fseek(g, r * sizeof(System), SEEK_SET);
                fread(&bufr, sizeof(System), 1, g);
            }
            if (l <= r)
            {
                swap(g, l, r);
                l++;
                r--;
            }
        }
        swap(g, left, r);
        if (left < r - 1)
            top = push(top, left, r - 1);
        if (r + 1 < right)
            top = push(top, r + 1, right);
    }
    fclose(g);
}
int vstafsort(const char *filename)
{
    FILE *p = proverka(filename, "rb+");
    if (!p)
        return 1;
    System buf, bufl;
    int j;
    fseek(p, 0, SEEK_END);

```

```

int n = ftell(p) / sizeof(System);
for (int i = 1; i < n; i++)
{
    fseek(p, i * sizeof(System), SEEK_SET);
    fread(&buf, sizeof(buf), 1, p);
    strcpy(buf1.vrempri, buf.vrempri);
    j = i - 1;
    while (j >= 0)
    {
        fseek(p, j * sizeof(System), SEEK_SET);
        fread(&buf1, sizeof(buf1), 1, p);
        if (strcmp(buf1.vrempri, buf.vrempri) <= 0)
            break;
        fseek(p, (j + 1) * sizeof(System), SEEK_SET);
        fwrite(&buf1, sizeof(buf1), 1, p);
        j -= 1;
    }
    fseek(p, (j + 1) * sizeof(System), SEEK_SET);
    fwrite(&buf, sizeof(System), 1, p);
}
fclose(p);
return 0;
}

int viborsort(const char *filename)
{
    FILE *p = proverka(filename, "rb+");
    if (!p)
        return 1;
    System min, s;
    int h;
    fseek(p, 0, SEEK_END);
    int n = ftell(p) / sizeof(System);
    for (int i = 0; i < n - 1; i++)
    {
        fseek(p, i * sizeof(System), SEEK_SET);
        fread(&min, sizeof(min), 1, p);
        h = i;
        for (int j = i + 1; j < n; j++)
        {
            fseek(p, j * sizeof(System), SEEK_SET);
            fread(&s, sizeof(s), 1, p);
            if (strcmp(s.vremotp, min.vremotp) < 0)
            {
                min = s;
                h = j;
            }
        }
        if (h == i)
            continue;
        swap(p, i, h);
    }
    fclose(p);
    return 0;
}

int create(const char *filename)
{
    int choose;
    FILE *p = proverka(filename, "wb");
    if (!p)
        return 1;
    cout << "Example: 25 Bmw London 12:45 13:09" << endl;
    cout << "Enter data : " << endl;
    System s;

```

```

while (true)
{
    cout << " number :" << endl;
    cin >> s.nomer;
    cin.ignore();
    cout << "Type" << endl;
    cin.getline(s.typeav, 10);
    cout << "Destination" << endl;
    cin.getline(s.punnaz, 15);
    cout << "Departure time" << endl;
    cin.getline(s.vremotp, 6);
    cout << "Arrival time" << endl;
    cin.getline(s.vrempri, 6);
    fwrite(&s, sizeof(s), 1, p);
    cout << "more?" << "\n";
    cout << "1-NO";
    cin >> choose;
    if (choose == 1)
        break;
}
fclose(p);
return 0;
}
int dobav(const char *filename)
{
    FILE *p = proverka(filename, "ab");
    if (!p)
        return 1;
    cout << "Adding new record: " << endl;
    System s;
    cout << "record number:" << endl;
    cin >> s.nomer;
    cin.ignore();
    cout << "Type of transport" << endl;
    cin.getline(s.typeav, 10);
    cout << "Destination :" << endl;
    cin.getline(s.punnaz, 15);
    cout << "Departure time" << endl;
    cin.getline(s.vremotp, 6);
    cout << "Arrival time" << endl;
    cin.getline(s.vrempri, 6);
    fwrite(&s, sizeof(s), 1, p);
    fclose(p);
    return 0;
}
int pros(const char *filename)
{
    FILE *p = proverka(filename, "rb");
    if (!p)
        return 1;
    System s;
    while (fread(&s, sizeof(s), 1, p) == 1)
    {
        cout << "Number:" << s.nomer << "| " << "Type:" << s.typeav << "| " <<
"Destination:" << s.punnaz << "| " << "Departure time:" << s.vremotp << "| "
<< "Arrival time:" << s.vrempri << endl;
    }
    fclose(p);
    return 0;
}
int vremotp(const char *filename)
{
    FILE *p = proverka(filename, "rb");

```

```

    if (!p)
        return 1;
    System s;
    char buf[6];
    cout << "Enter departure time" << '\n';
    cin.getline(buf, 6);
    while (fread(&s, sizeof(s), 1, p) == 1)
    {
        if (strcmp(s.vremotp, buf) == 0)
        {
            cout << "Found:" << '\n';
            cout << "Destination:" << s.punnaz << "| " << "Number:" << s.nomer
<< "| " << "Type:" << s.typeav << "| " << "Arrival time:" << s.vrempri << endl;
        }
    }
    fclose(p);
    return 0;
}
int punnaz(const char *filename, const char *bufname)
{
    copy_file(filename, bufname);
    quicksorts(bufname);
    FILE *p = proverka(bufname, "rb+");
    if (!p)
        return 1;
    char arrr[15];
    cout << "Enter destination" << '\n';
    cin.getline(arrr, sizeof(arrr));
    fseek(p, 0, SEEK_END);
    int left = 0, i = 0, j = 0;
    int right = ftell(p) / sizeof(System) - 1;
    System s;
    while (left <= right)
    {
        int mid = (right + left) / 2;
        fseek(p, mid * sizeof(System), SEEK_SET);
        fread(&s, sizeof(System), 1, p);
        if (strcmp(s.punnaz, arrr) == 0)
        {
            cout << "Result: " << '\n';
            cout << "Number:" << s.nomer << "| " << "Type:" << s.typeav << "|
" << "Departure time:" << s.vremotp << "| " << "Arrival time:" << s.vrempri <<
endl;

            i = mid - 1;
            j = mid + 1;
            while (i >= left)
            {
                fseek(p, i * sizeof(System), SEEK_SET);
                fread(&s, sizeof(System), 1, p);
                if (strcmp(s.punnaz, arrr) == 0)
                {
                    cout << "Number:" << s.nomer << "| " << "Type:" << s.typeav
<< "| " << "Departure time:" << s.vremotp << "| " << "Arrival time:" << s.vrempri
<< endl;
                }
                i--;
            }
            fseek(p, j * sizeof(System), SEEK_SET);
            while (j <= right)
            {
                fread(&s, sizeof(System), 1, p);
                if (strcmp(s.punnaz, arrr) == 0)
                {

```

```

        cout << "Number:" << s.nomer << "| " << "Type:" << s.typeav
<< "| " << "Departure time:" << s.vremotp << "| " << "Arrival time:" << s.vrempri
<< endl;
    }
    j++;
}
break;
}
else if (strcmp(s.punnaz, arrr) < 0)
    left = mid + 1;
else
    right = mid - 1;
}
fclose(p);
return 0;
}
int poiskpriz(const char *filename, const char *bufname, const char *filenamh)
{
    copy_file(filename, bufname);
    viborsort(bufname);
    FILE *f = proverka(bufname, "rb+");
    if (!f)
        return 1;
    char bufe[15];
    char vrem[6];
    System s;
    int choose;
    cout << "Where show\n1-console\n2-file\n";
    cin >> choose;
    cin.ignore();
    cout << "Enter destination " << endl;
    cin.getline(bufe, sizeof(bufe));
    cout << "Enter time" << endl;
    cin.getline(vrem, 6);
    switch (choose)
    {
    case 1:
    {
        while (fread(&s, sizeof(System), 1, f) == 1)
        {
            if (strcmp(s.vrempri, vrem) <= 0 && strcmp(bufe, s.punnaz) == 0)
                cout << "Type: " << s.typeav << "\nDeparture: " << s.vremotp
<< "\n";
        }
        break;
    }
    case 2:
    {
        FILE *k = proverka(filenamh, "w");
        if (!k)
            return 1;
        while (fread(&s, sizeof(System), 1, f) == 1)
        {
            if (strcmp(s.vrempri, vrem) <= 0 && strcmp(bufe, s.punnaz) == 0)
                fprintf(k, "Type: %s Departure: %s\n", s.typeav, s.vremotp);
        }
        fclose(k);
        break;
    }
    default:
        cout << "Not correct choose" << endl;
        fclose(f);
        return 1;
    }
}

```

```

    }
    fclose(f);
    return 0;
}
int staticiok(const char *filename, const char *filenamh)
{
    FILE *p = proverka(filename, "rb");
    if (!p)
        return 1;
    char typebuf[100];
    char vremya[6];
    int y = 0;
    int choose;
    System s;
    cout << "Where show\n1-console\n2-file\n";
    cin >> choose;
    cin.ignore();
    cout << "Enter type and minimum departure time: " << endl;
    cin.getline(typebuf, sizeof(typebuf));
    cin.getline(vremya, 6);
    switch (choose)
    {
    case 1:
    {
        while (fread(&s, sizeof(System), 1, p) == 1)
        {
            if (strcmp(typebuf, s.typeav) == 0 && strcmp(s.vremotp, vremya) >
0)
            {
                cout << "Type: " << s.typeav << "\nDeparture: " << s.vremotp
<< "\nDestination: " << s.punnaz << "\n\n";
                y++;
            }
        }
        cout << "The number of possible paths: " << y << '\n';
        break;
    }
    case 2:
    {
        FILE *g = proverka(filenamh, "w");
        if (!g)
        {
            fclose(p);
            return 1;
        }
        while (fread(&s, sizeof(System), 1, p) == 1)
        {
            if (strcmp(typebuf, s.typeav) == 0 && strcmp(s.vremotp, vremya) >
0)
            {
                fprintf(g, "Type: %s Departure: %s Destination: %s\n", s.typeav,
s.vremotp, s.punnaz);
                y++;
            }
        }
        fprintf(g, "\nThe number of possible paths: %d\n", y);
        fclose(g);
        break;
    }
    default:
        cout << "Not correct choose" << endl;
        fclose(p);
        return 1;
    }
}

```

```

    }
    fclose(p);
    return 0;
}
int main()
{
    char filename[256] = "sigma.bin";
    const char *bufname = "buf.bin";
    const char *filetxt = "result.txt";
    int choose;
    while (true)
    {
        cout << "1. Create file " << endl;
        cout << "2. View all records" << endl;
        cout << "3. Search by departure time" << endl;
        cout << "4. Search by destination" << endl;
        cout << "5. Search (destination + time)" << endl;
        cout << "6. Quick sort by destination" << endl;
        cout << "7. Selection sort by departure time" << endl;
        cout << "8. Insertion sort by arrival time" << endl;
        cout << "9. Statistics" << endl;
        cout << "10. Delete" << endl;
        cout << "11. Added" << endl;
        cout << "12. Change" << endl;
        cout << "13. New file" << endl;
        cout << "0. Exit" << endl;
        cout << "Your choice: ";
        cin >> choose;
        cin.ignore();

        switch (choose)
        {
            case 1:
                create(filename);
                break;
            case 2:
                pros(filename);
                break;
            case 3:
                vremotp(filename);
                break;
            case 4:
                punnaz(filename, bufname);
                break;
            case 5:
                poiskpriz(filename, bufname, filetxt);
                break;
            case 6:
                quicksorts(filename);
                pros(filename);
                break;
            case 7:
                viborsort(filename);
                break;
            case 8:
                vstafsort(filename);
                break;
            case 9:
                staticiok(filename, filetxt);
                break;
            case 10:
                ydalen(filename);
                break;
        }
    }
}

```

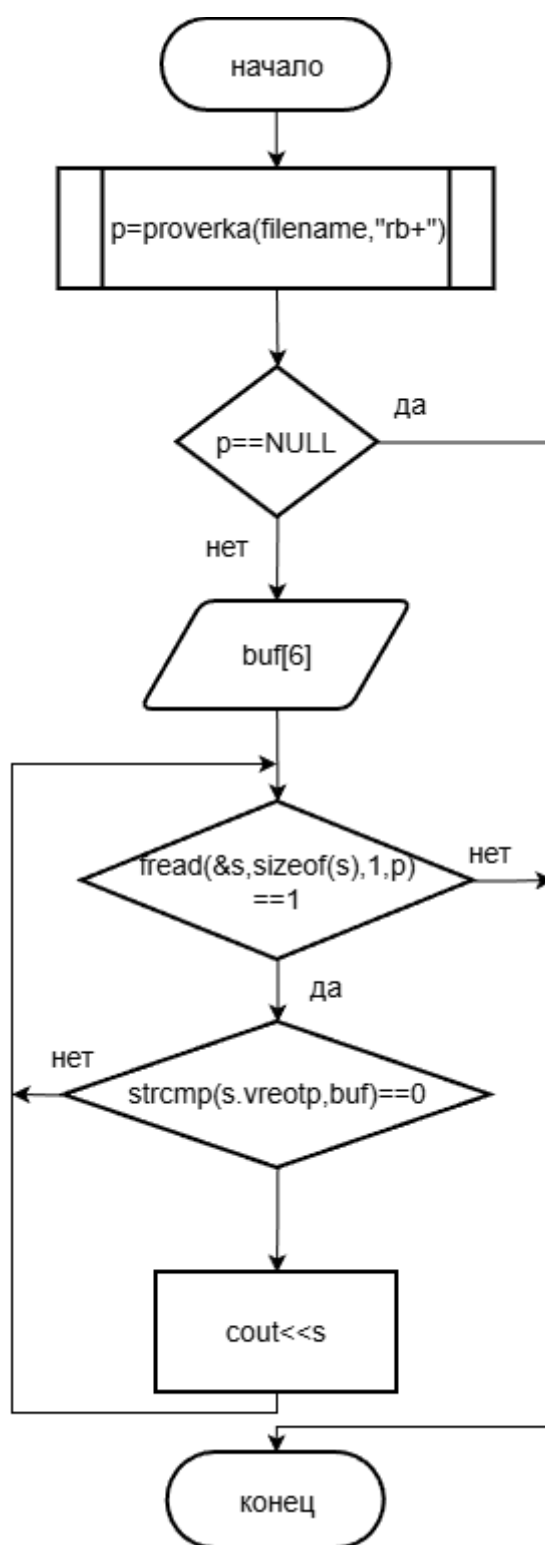


```

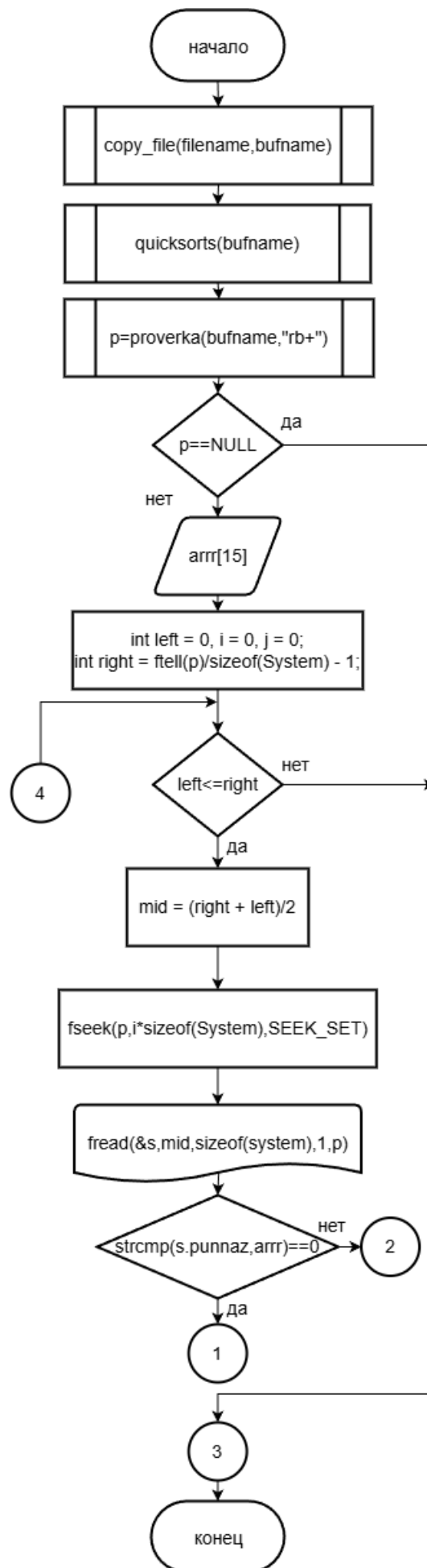
    case 11:
        dobav(filename);
        break;
    case 12:
        izme(filename);
        break;
    case 13:
        int buf;
        cout << "1-Work with old\n2-Work with new\nYour choose: ";
        cin >> buf;
        cin.ignore();
        if (buf == 1)
        {
            cout << "Enter filename: \n";
            cin.getline(filename, 256);
        }
        else
        {
            cout << "Enter new filename: \n";
            cin.getline(filename, 256);
            create(filename);
        }
        break;
    case 0:
        cout << "Exit" << endl;
        return 0;
    default:
        cout << "Not correct choose" << endl;
        break;
}
}
return 0;
}

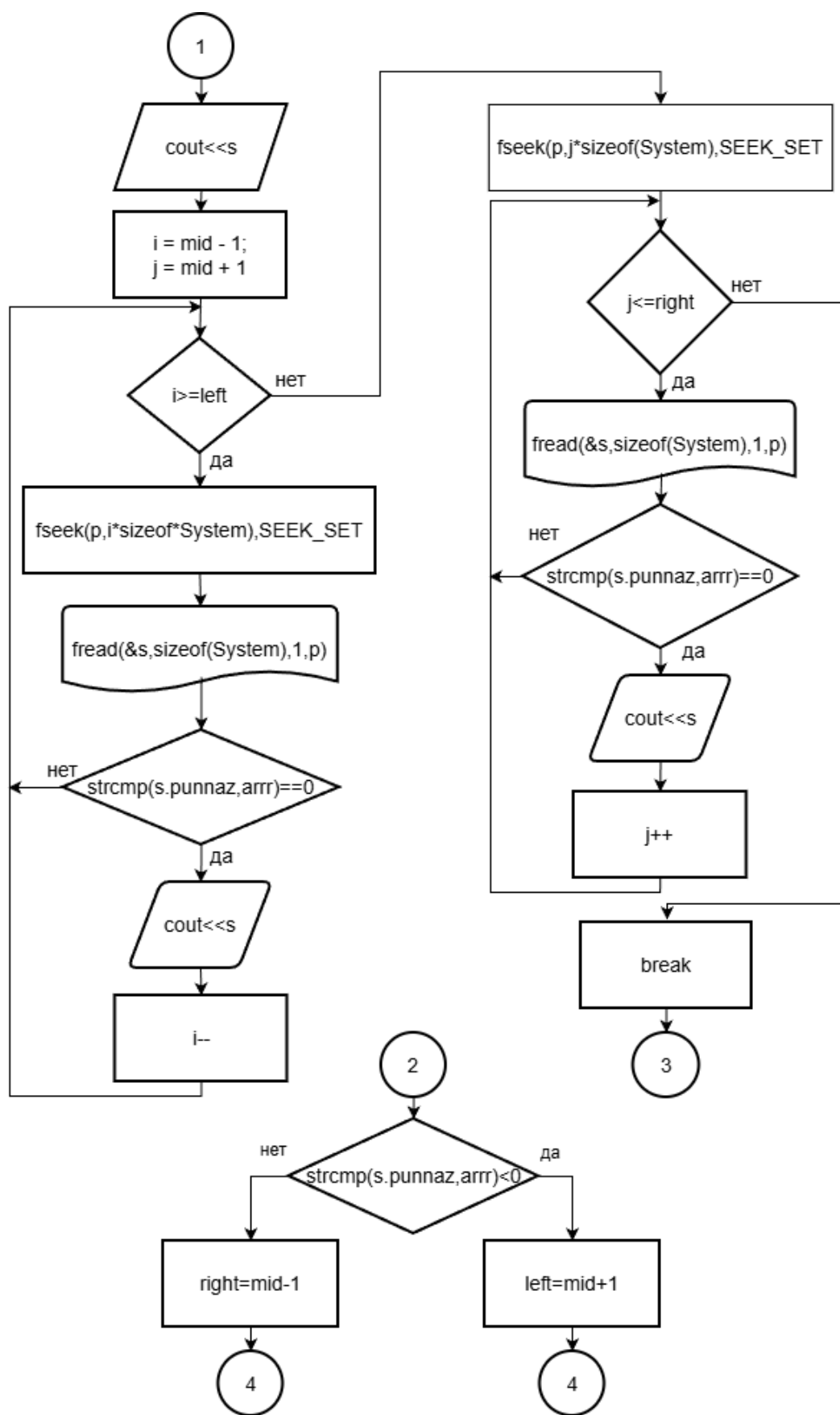
```

ПРИЛОЖЕНИЕ Б
(обязательное)
Блок-схемы функций

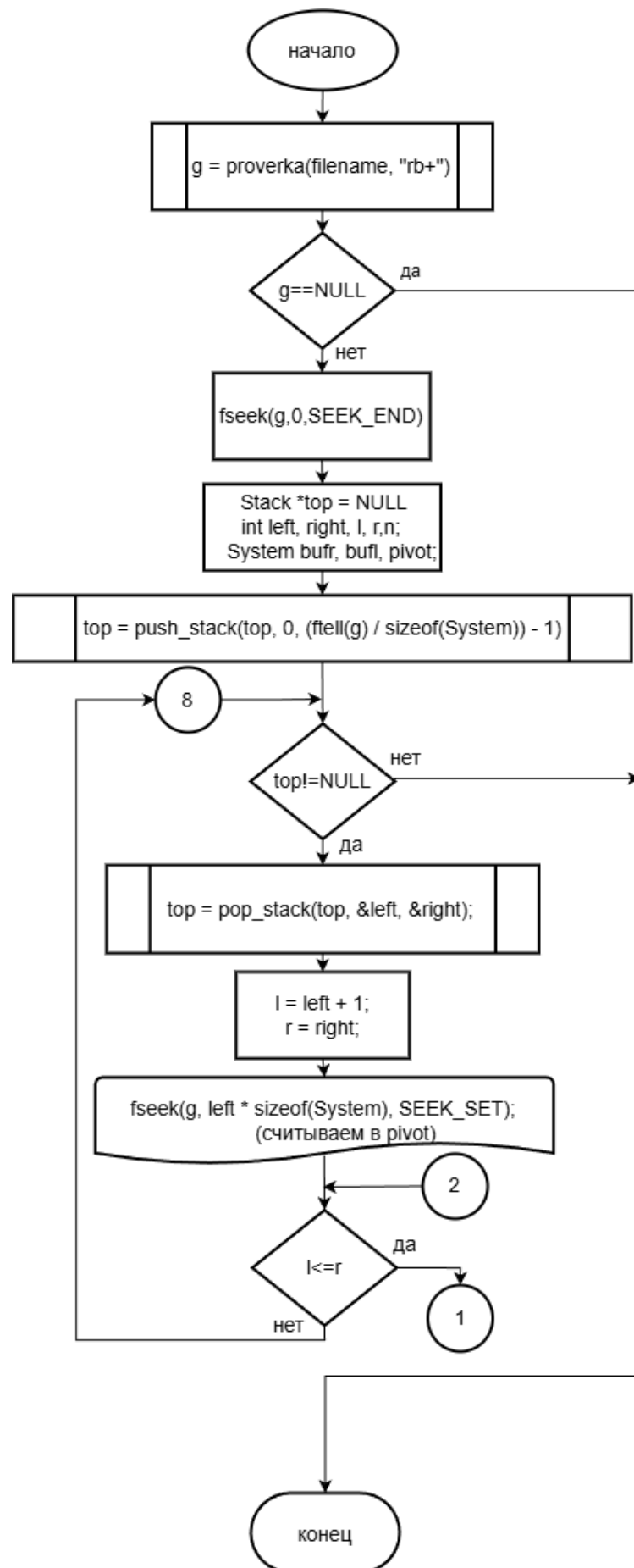


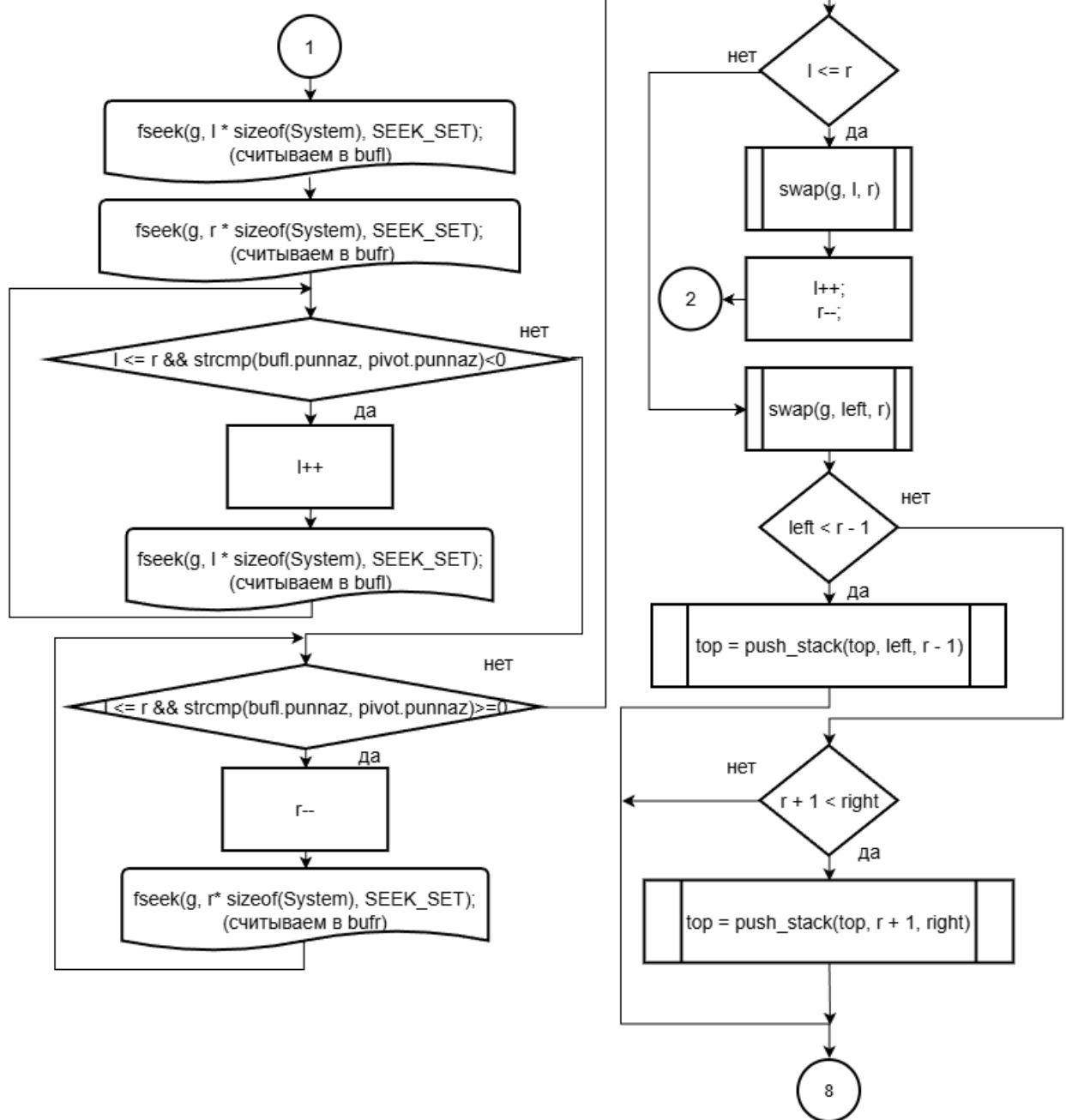
Б.1 – Блок-схема функции `vremotp()`



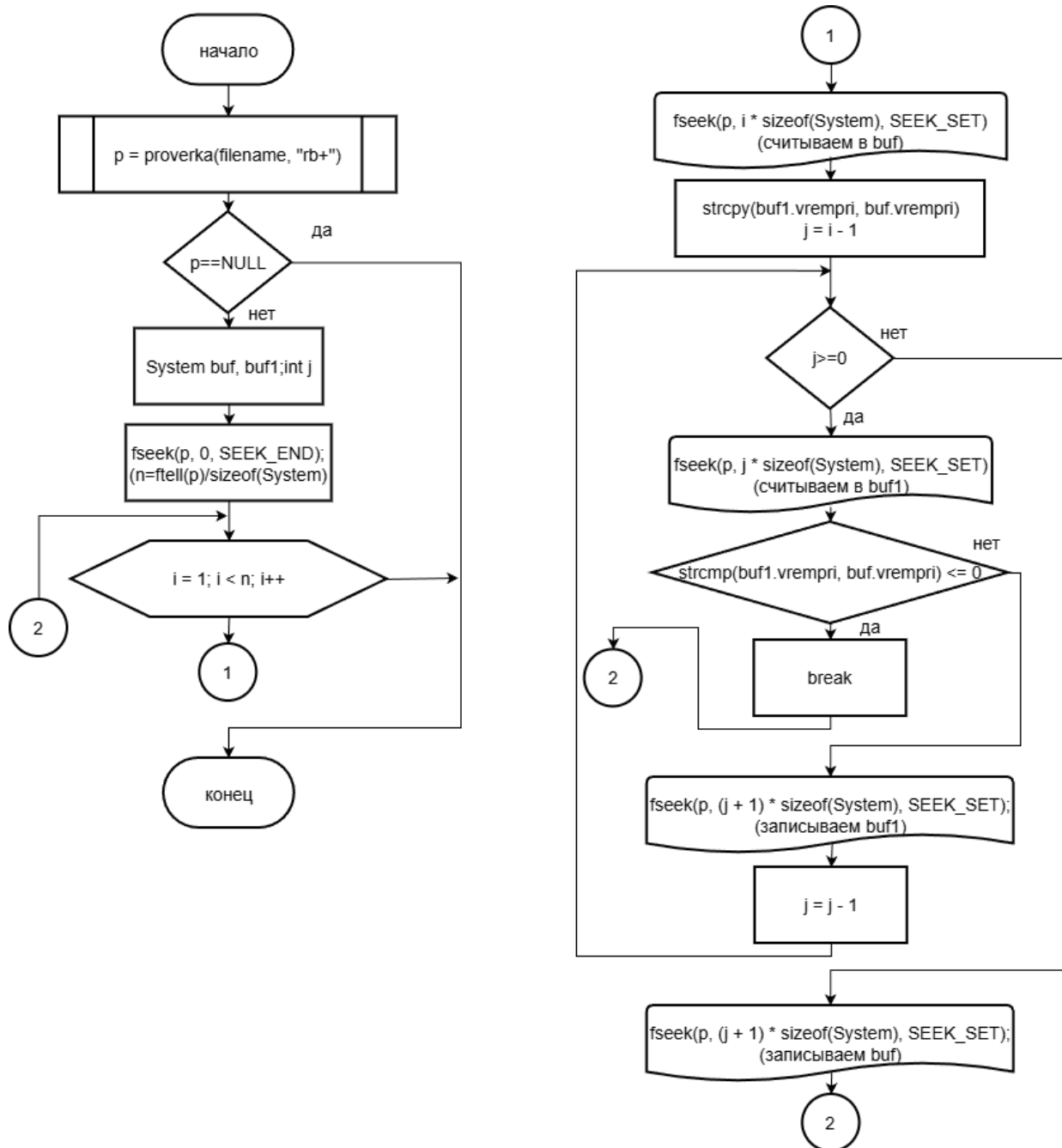


Б.2 - Блок - схема функции punnaz()

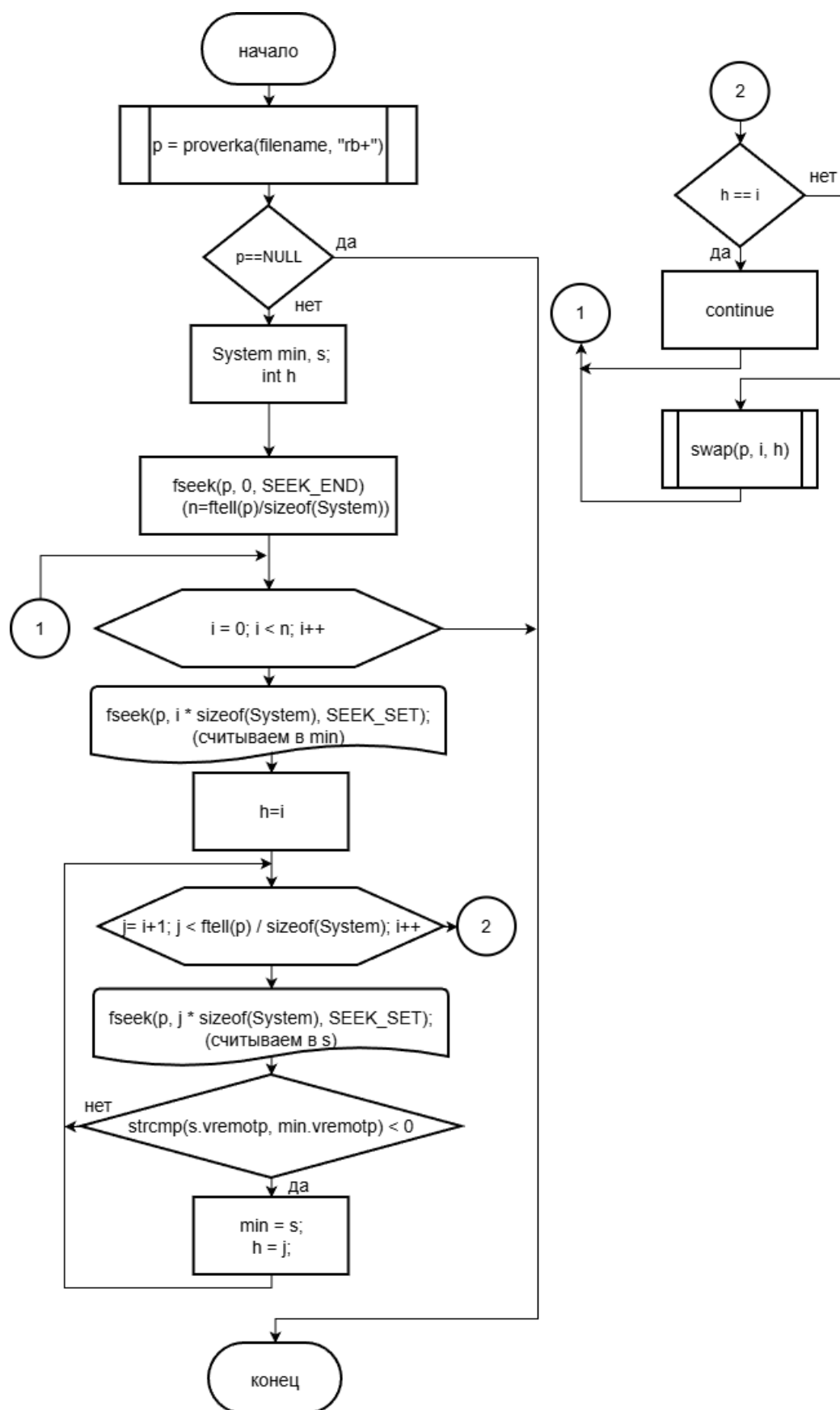




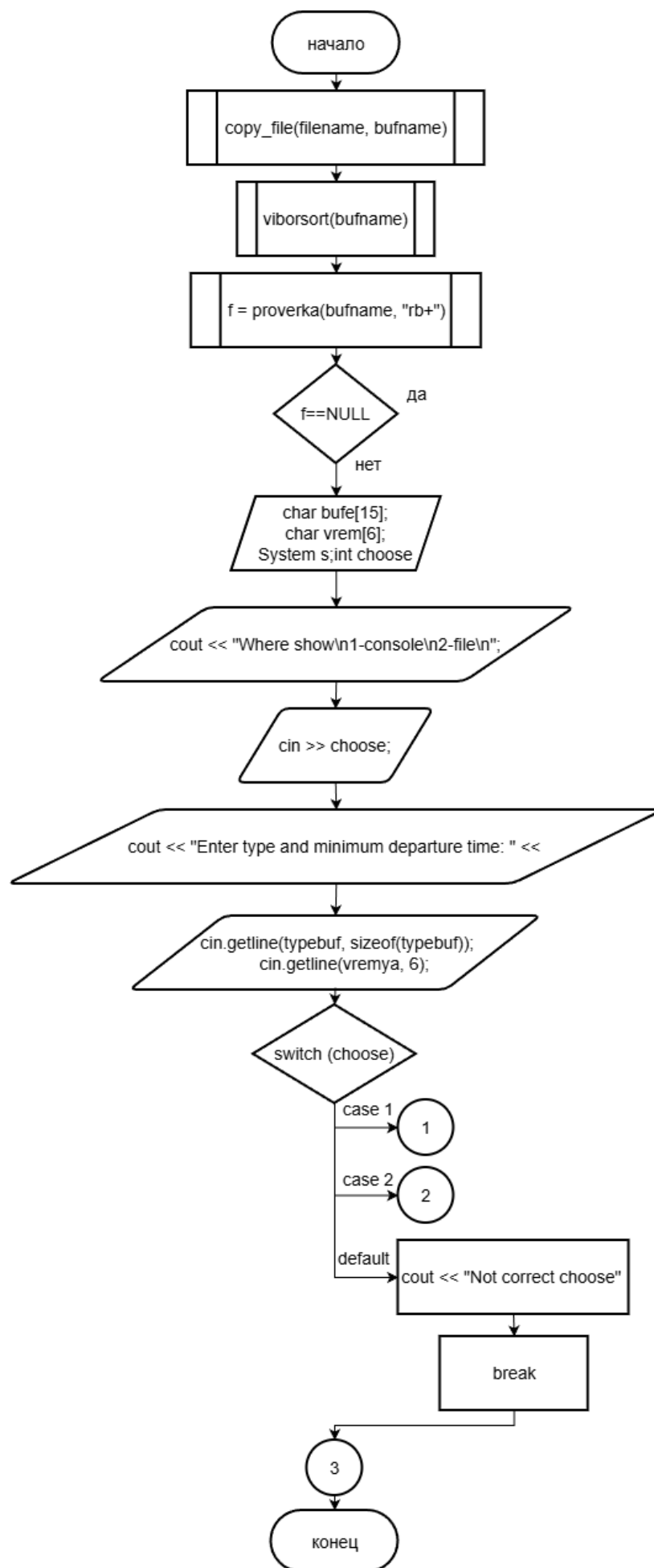
Б.3 – Блок – схема функции quicksort()

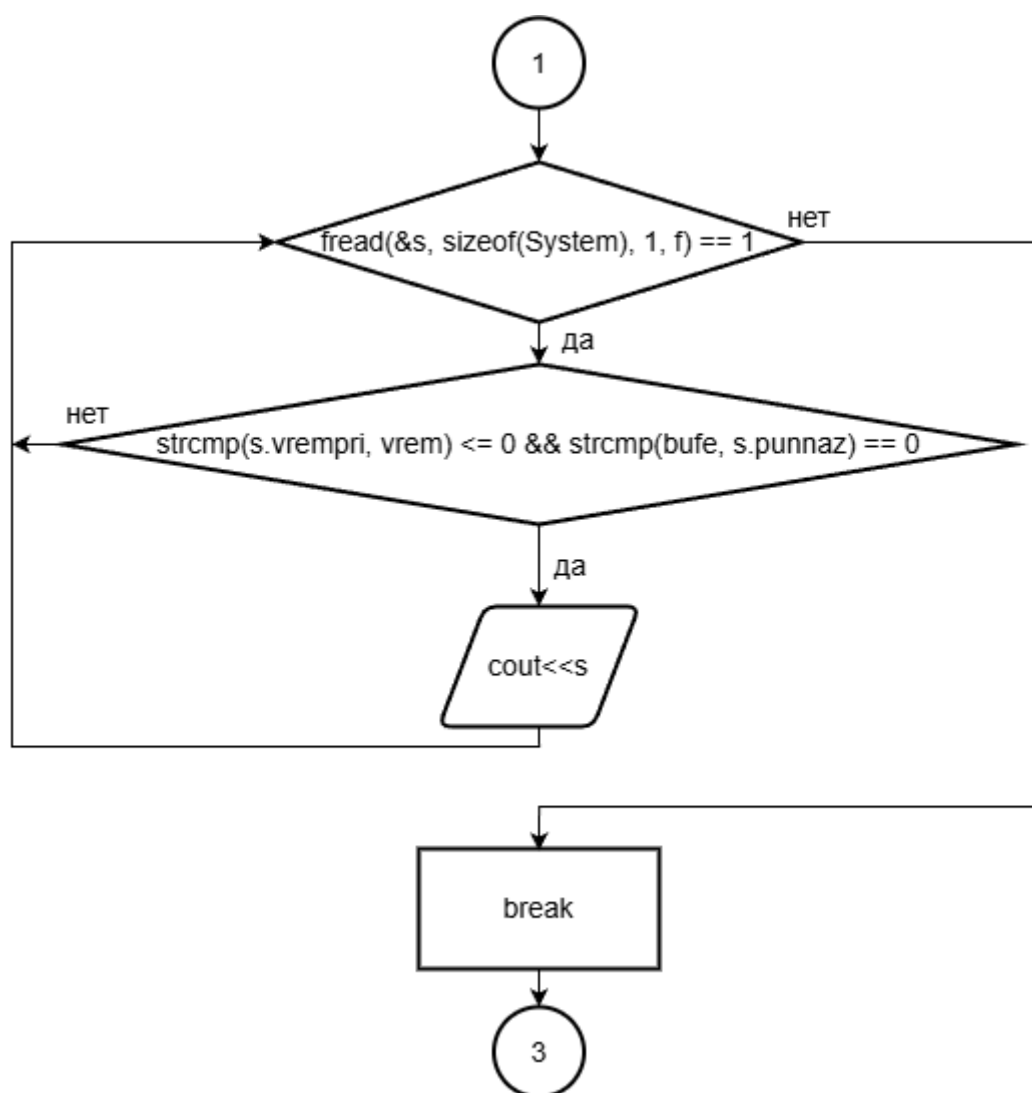


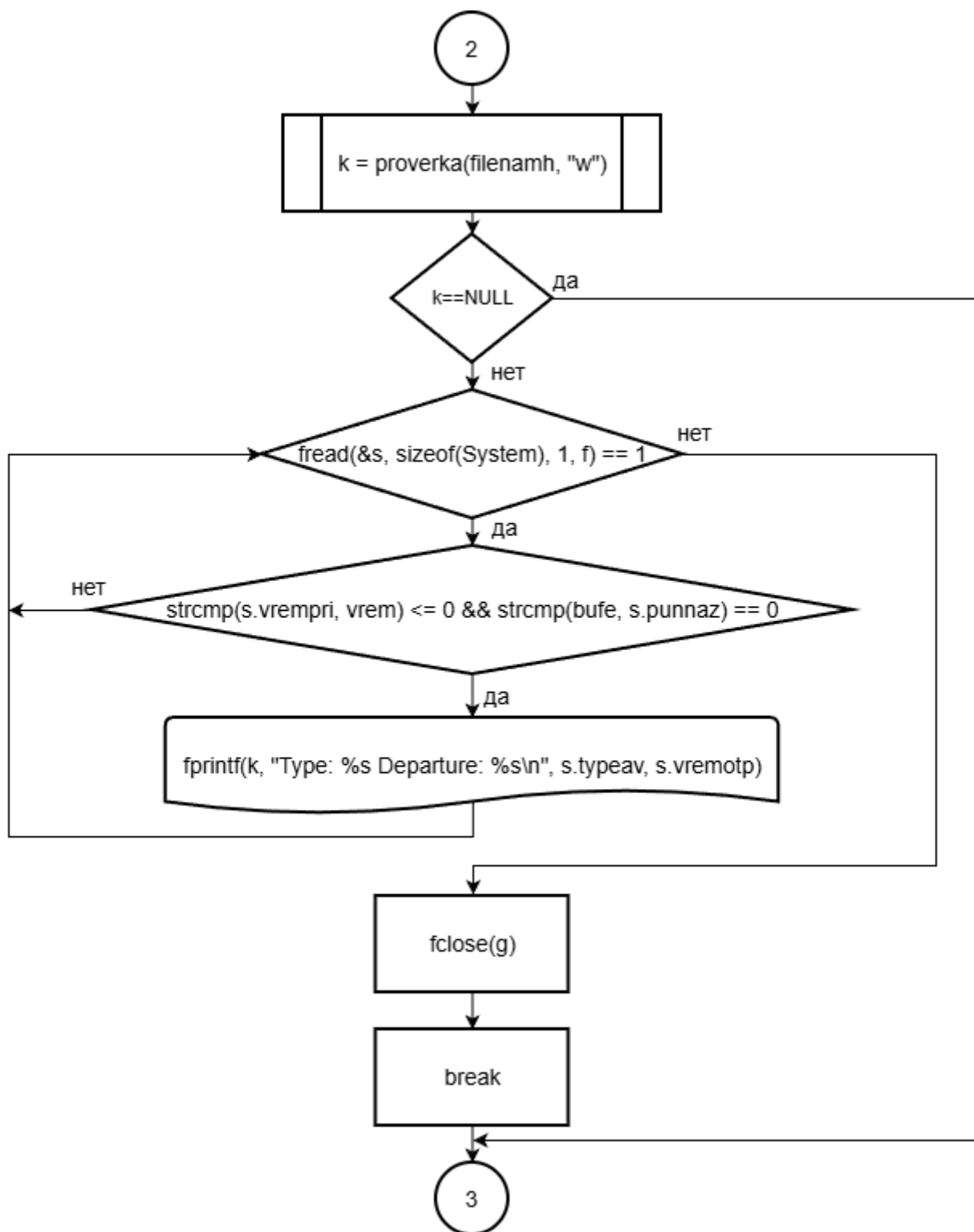
Б.4 – Блок – схема функции `vstafsort()`



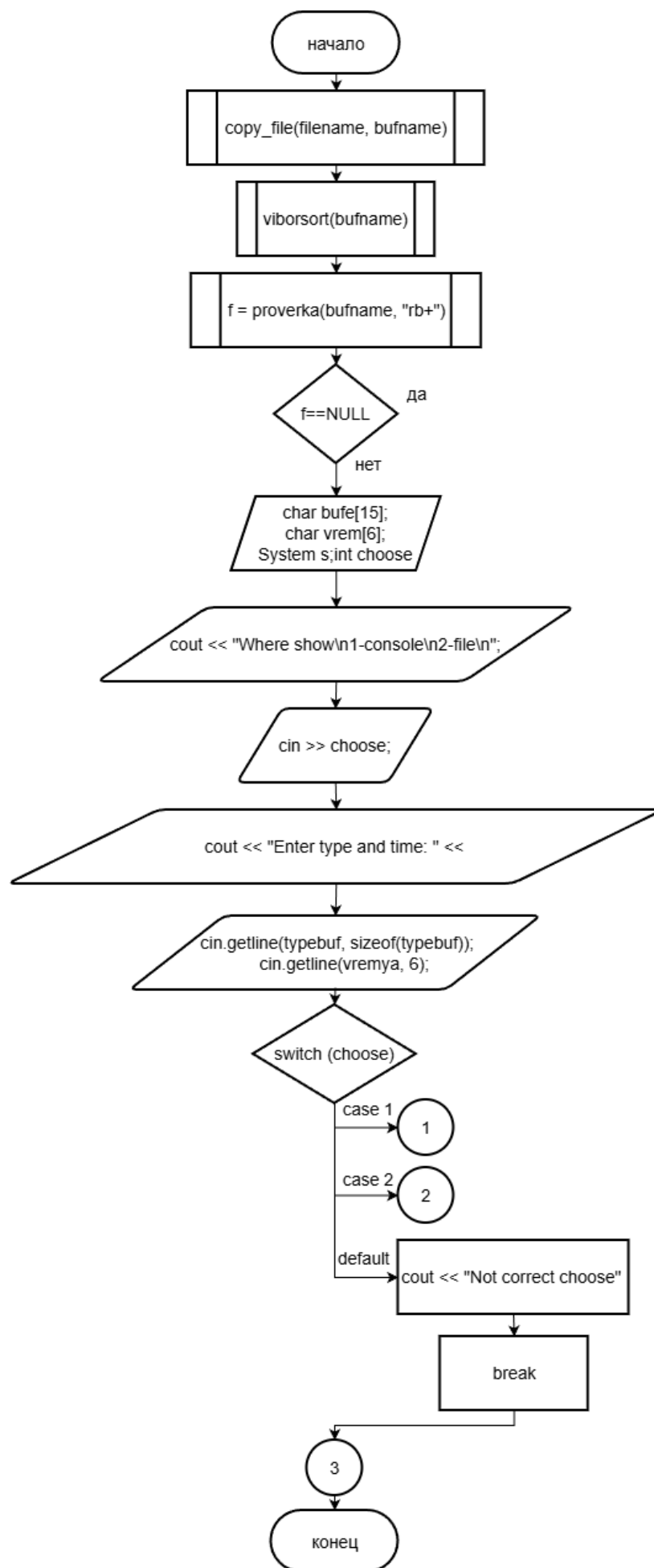
Б.5 – Блок – схема функции viborsort()

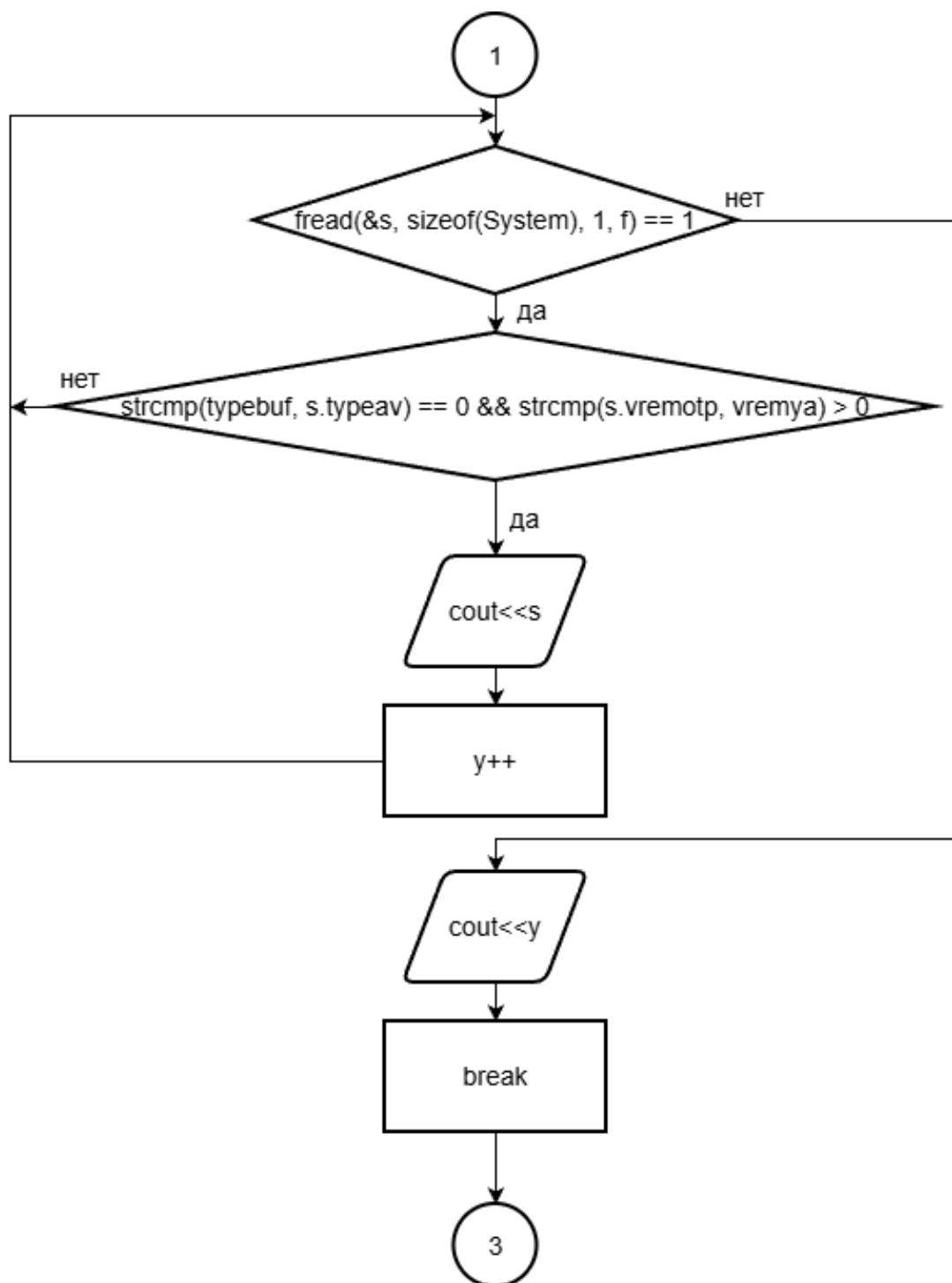


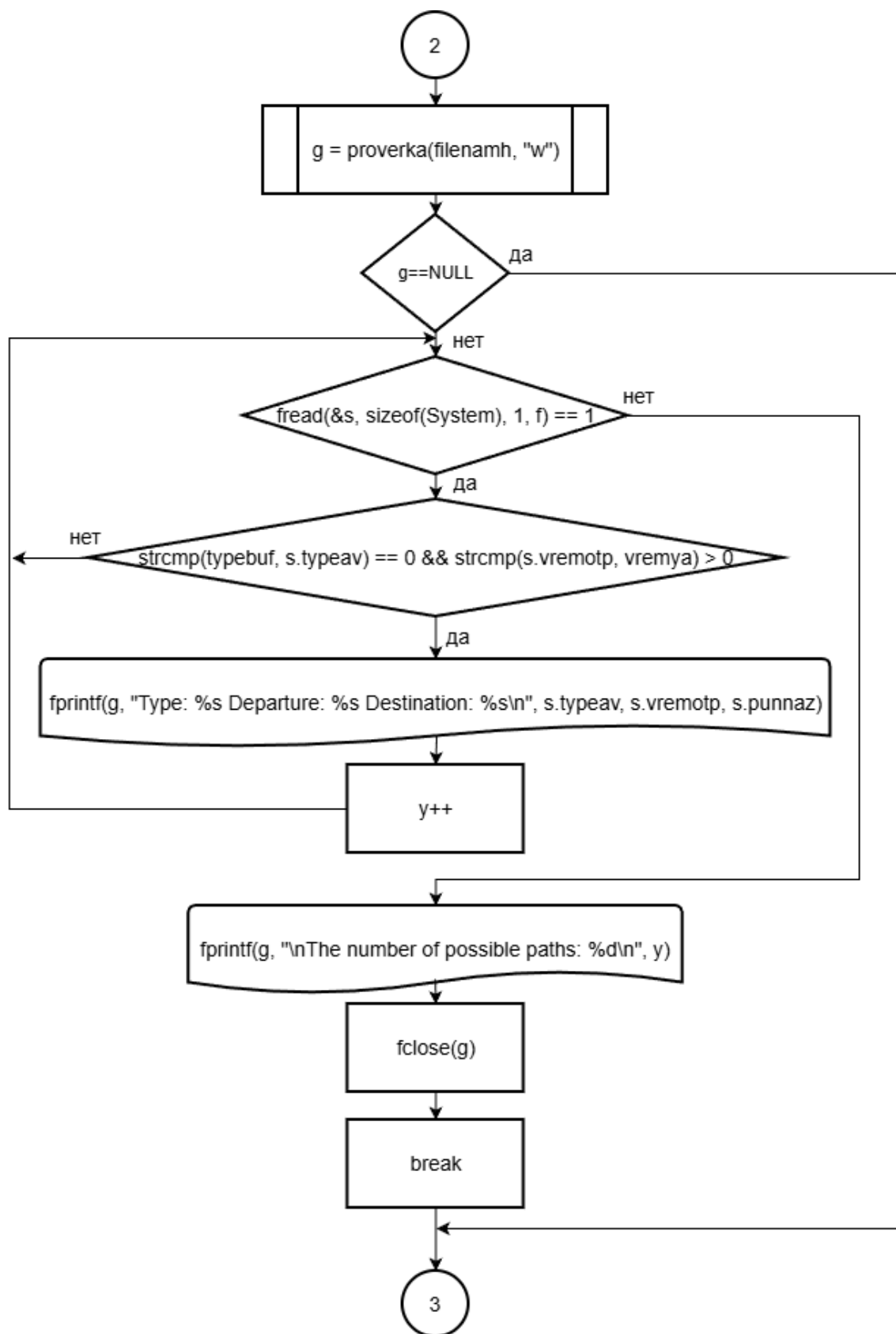




Б.6 – Блок – схема функции poiskpriz()







Б.7 – Блок – схема функции staticiok()

ВЕДОМОСТЬ